

Thorpe Desktop — Patent Attorney Submission Package

Prepared for: Patent counsel review and filing strategy
Product: Thorpe Desktop (cross-platform IT support application)
Core invention: Jonathan — autonomous AI technician with local repair orchestration
Codebase reference: /workspace (Thorpe-Workforce repository, Desktop branch)
Document date: June 27, 2026
Software version referenced: 1.1.0 (see package.json, src-tauri/Cargo.toml)

Important notice (read first)

This package is a **technical invention disclosure** prepared from the Thorpe Desktop source code and product design. It is **not legal advice**, does not constitute filed patent claims, and does not establish priority dates. Your patent attorney must:

- Conduct a formal prior-art and patentability search
- Draft claims appropriate to your jurisdiction (USPTO, EPO, etc.)
- Confirm inventorship and ownership with the client
- Advise on provisional vs. non-provisional, PCT, and continuation strategy
- Review export control and open-source implications before filing

Package contents

#	Document	Purpose
01	[01-INVENTION-DISCLOSURE-JONATHAN.md](./01-INVENTION-DISCLOSURE-JONATHAN.md)	Primary invention disclosure — how Jonathan works, step-by-step
02	[02-PATENT-CANDIDATE-INVENTORY.md](./02-PATENT-CANDIDATE-INVENTORY.md)	Recommended patent applications (utility + optional design)
03	[03-SYSTEM-ARCHITECTURE-AND-FLOWS.md](./03-SYSTEM-ARCHITECTURE-AND-FLOWS.md)	Architecture diagrams, data structures, sequence flows
04	[04-CLAIM-SEEDS-FOR-ATTORNEY.md](./04-CLAIM-SEEDS-FOR-ATTORNEY.md)	Seed independent/dependent claims for attorney drafting
05	[05-PRIOR-ART-DIFFERENTIATION.md](./05-PRIOR-ART-DIFFERENTIATION.md)	Known art categories and differentiation arguments

06	[06-INVENTOR-AND-ASSIGNMENT-SHEET.md](./06-INVENTOR-AND-ASSIGNMENT-SHEET.md)	Inventor information template and assignment checklist
07	[07-EXECUTIVE-BRIEF-ONE-PAGE.md](./07-EXECUTIVE-BRIEF-ONE-PAGE.md)	Non-technical one-page summary for counsel
08	[08-ATTORNEY-TRANSMITTAL-LETTER.md](./08-ATTORNEY-TRANSMITTAL-LETTER.md)	Cover letter template (fill in and send)
09	[09-FIGURE-SCREENSHOT-GUIDE.md](./09-FIGURE-SCREENSHOT-GUIDE.md)	Figure numbering, screenshots, diagram conversion
—	[dist/Thorpe-Desktop-Patent-Package.pdf](./dist/Thorpe-Desktop-Patent-Package.pdf)	Combined PDF (run <code>bash scripts/build-patent-package.sh</code>)
—	[screenshots/](./screenshots/)	UI captures for specification figures

Executive summary for counsel

Thorpe Desktop is a local-first desktop application (Windows, macOS, Linux) that embeds **Jonathan**, an AI technician agent. Unlike conventional chatbot IT assistants that recommend manual steps, Jonathan:

1. **Plans** repairs using rules and/or LLM JSON planning
2. **Executes** OS-level diagnostic and repair actions locally **before** generating user-facing narrative
3. **Verifies** mutating repairs with a follow-up health scan
4. **Gates** mutating actions behind user confirmation while auto-running safe diagnostics
5. **Operates offline** for core diagnostics (connectivity suite, repair engine, local narrative)
6. **Integrates** proactive monitoring (Watchdog) that hands off pre-planned incidents into the same pipeline
7. **Restricts** autonomous tool permissions via cryptographically signed repair packs
8. **Separates** planning LLM and narration LLM with independent fallbacks

The recommended filing strategy (detailed in Document 02) centers on **one broad system/method patent** on the execute-then-narrate autonomous agent architecture, supplemented by **narrower patents** on connectivity playbook fusion, signed repair-pack gating, watchdog handoff, and post-repair verification loops.

Key source files (for examiner / code deposit)

Subsystem	Primary paths
Chat entry & narrative	<code>src-tauri/src/ai/mod.rs</code>
Incident orchestration	<code>src-tauri/src/agent/mod.rs</code>
LLM/rule planner	<code>src-tauri/src/agent/planner.rs</code>
Repair engine	<code>src-tauri/src/repairs/mod.rs, planner.rs</code>

Repair pack signing	src-tauri/src/repairs/pack_signing.rs, packs.rs
Offline connectivity	src-tauri/src/connectivity/mod.rs
Proactive watchdog	src-tauri/src/watchdog/mod.rs
Evidence collection	src-tauri/src/evidence/mod.rs
Enterprise AI policy	src-tauri/src/enterprise_ai/mod.rs
Licensing / features	src-tauri/src/licensing/mod.rs
UI — Jonathan chat	src/pages/JonathanAssistant.tsx
UI — Watchdog handoff	src/lib/watchdog.ts, src/components/layout/AppLayout.tsx

Suggested next steps for attorney

1. Review Documents 01–04 and confirm inventorship (Document 06).
2. Run prior-art search on: autonomous IT repair agents, LLM tool execution, desktop health monitors, MSP PSA integrations.
3. Decide provisional filing date and claim scope for **Patent A** (core Jonathan orchestration).
4. Evaluate whether to file **Patent B–F** as continuations or separate applications.
5. Request code listing or appendix if filing requires software-related enablement (Alice/Mayo analysis for US).
6. Coordinate trademark (Jonathan, Thorpe) separately from utility patents.

Building the package

From the repository root:

```
bash
npm run patent:package
```

Or directly:

```
bash
bash scripts/build-patent-package.sh
```

Requirements: Node.js 18+, npm install dependencies, Google Chrome or Microsoft Edge installed.

Windows (Command Prompt or PowerShell):

```
cmd
cd path\to\Thorpe-Workforce
```

```
npm install  
npm run patent:package
```

If Chrome is in a non-standard location, set the browser path first:

```
cmd  
set CHROME_PATH=C:\Program Files\Google\Chrome\Application\chrome.exe  
npm run patent:package
```

macOS / Linux:

```
bash  
npm run patent:package
```

optional: `CHROME_PATH=/path/to/chrome npm run patent:package`

The script builds the frontend, starts a Vite preview server, captures six UI screenshots with Puppeteer, and exports docs/patents/dist/Thorpe-Desktop-Patent-Package.html plus PDF when Chrome print-to-PDF succeeds.

Outputs:

Path	Description
docs/patents/screenshots/*.png	Figure captures (dashboard, Jonathan, watchdog, repairs)
docs/patents/dist/Thorpe-Desktop-Patent-Package.html	Combined attorney package
docs/patents/dist/Thorpe-Desktop-Patent-Package.pdf	Print-ready PDF (if Chrome available)

Submit the docs/patents/ folder (markdown sources + dist/ + screenshots/) to patent counsel.

Contact / assignment

Complete Document 06 before filing.

Field	Value
Applicant / assignee	<u>[Company legal name]</u>
Address	<u>[To be completed]</u>
Correspondence	<u>[To be completed]</u>
Inventor(s)	<u>[See Document 06]</u>

Thorpe Desktop / Jonathan — One-Page Executive Brief for Patent Counsel

Product: Thorpe Desktop · **Agent:** Jonathan · **Date:** June 27, 2026

What we invented

Jonathan is an AI technician embedded in a desktop app that **fixes computers locally**, not just advises. When a user says “Wi-Fi isn’t working” or “my PC is slow,” Jonathan:

1. Diagnoses the machine on-device (no cloud required for core operation)
2. **Runs repairs automatically** (safe diagnostics) and asks permission before system-changing actions
3. **Re-scans** to verify the fix improved health score
4. **Then** explains what it did—in plain language or via optional cloud AI

Core novelty: Repairs execute **before** the AI writes its reply. The AI cannot invent fixes it did not perform.

Why it matters commercially

Problem today	Jonathan approach
Chatbots give manual steps	Executes OS-level repairs locally
Cloud-only AI support	Works offline for diagnostics + local narrative
No proof a “fix” worked	Before/after health verification in every response
Unsafe autonomous agents	Mutating actions require explicit user approval
MSP tools need IT staff	End-user conversational interface

Markets: Consumer PC support, SMB without IT staff, MSP assist (PSA webhooks), enterprise with governed AI (role/budget controls).

What to patent (priority)

Priority	Application	One-line description
P0	A — Core orchestration	Execute-then-narrate autonomous IT incident pipeline on client device
P0	B — Repair packs	Cryptographically signed allowlist gates which OS tools AI may run
P0	C — Offline connectivity	Layered network diagnostics fused into agent; no cloud required
P1	D — Watchdog handoff	Background CPU/memory/disk monitor hands alerts into same agent pipeline
P1	E — Dual LLM	Separate planner LLM and narrator LLM with independent fallbacks

Full detail: 02-PATENT-CANDIDATE-INVENTORY.md

Technical proof (reduction to practice)

- **Shipped in code:** Thorpe Desktop v1.1.0 (Rust/Tauri + React)
- **Platforms:** Windows, macOS, Linux
- **Key module:** `orchestrate_incident` in `src-tauri/src/agent/mod.rs`
- **Tests:** 70+ frontend, 40+ Rust unit tests

Urgency / disclosure risk

- Source and releases on **GitHub** (public) — counsel must confirm bar dates and recommend **provisional filing ASAP** on Patent A if not already filed.
- Complete `06-INVENTOR-AND-ASSIGNMENT-SHEET.md` before filing.

What we need from counsel

1. Prior-art search on Patents A–E
2. Provisional filing recommendation and timeline
3. Alice (§101) analysis for software method claims
4. Inventorship and assignment to operating company
5. Trademark clearance: **Thorpe, Jonathan**

Full package: `docs/patents/00-PATENT-PACKAGE-INDEX.md`

Transmittal Letter — Patent Counsel

[Date]

[Patent Attorney Name]

[Law Firm]

[Address]

Re: Invention Disclosure — *Thorpe Desktop / Jonathan Autonomous AI Technician*

Applicant: [Company Legal Name]

Confidential — Attorney-Client Privileged

Dear [Attorney Name],

We are submitting a complete invention disclosure package for **Thorpe Desktop**, a cross-platform IT support application, and its embedded AI technician **Jonathan**. We request your evaluation for utility patent protection in the United States (and advise on international strategy).

Nature of the invention

Jonathan is not a conventional support chatbot. It is a **local-first autonomous incident orchestration system** that:

- Plans repairs using rules and/or a structured LLM planner
- **Executes operating-system-level diagnostics and repairs on the client device before generating any user-facing AI narrative**
- Requires explicit user approval for mutating (system-changing) actions
- Verifies successful repairs with a follow-up health scan
- Operates core diagnostics **offline**, including a layered network connectivity suite
- Restricts autonomous tool execution via **cryptographically signed repair packs**
- Integrates **proactive background monitoring** (CPU, memory, disk) with handoff into the same repair pipeline

We believe the primary protectable concept is the **execute-then-narrate** architecture combined with signed tool allowlisting and offline diagnostic fusion.

Enclosed materials

Item	Location in package
Package index and executive summary	00-PATENT-PACKAGE-INDEX.md
Technical invention disclosure	01-INVENTION-DISCLOSURE-JONATHAN.md
Recommended patent applications (A–H)	02-PATENT-CANDIDATE-INVENTORY.md
Architecture figures (Mermaid)	03-SYSTEM-ARCHITECTURE-AND-FLOWS.md
Seed claims for drafting	04-CLAIM-SEEDS-FOR-ATTORNEY.md
Prior-art differentiation notes	05-PRIOR-ART-DIFFERENTIATION.md
Inventor / assignment sheet	06-INVENTOR-AND-ASSIGNMENT-SHEET.md (to be completed)
One-page executive brief	07-EXECUTIVE-BRIEF-ONE-PAGE.md
UI screenshots	screenshots/
Combined PDF (if generated)	dist/Thorpe-Desktop-Patent-Package.pdf

Source code reference: repository **Thorpe-Workforce**, branch/tag v1.1.0, paths listed in Document 00.

Requested services

1. **Prior-art search** covering conversational IT agents, RMM/remediation tools, LLM tool-use systems, and offline network diagnostics.
2. **Patentability opinions** on proposed Applications **A through E** (see Document 02).
3. **Filing strategy** including whether to file a **provisional** application immediately for Application A given

- public GitHub disclosure.
- 4. **Claim drafting** from seed claims in Document 04; advise on combining vs. separating applications.
 - 5. **§101 (Alice) analysis** for software/method claims.
 - 6. **Inventorship and assignment** review upon return of completed Document 06.
 - 7. **Trademark** preliminary clearance for *Thorpe* and *Jonathan* in Classes 9 and 42.

Priority applications (our recommendation)

- **Application A (P0):** System and method for local-first autonomous IT incident resolution with execute-then-narrate AI assistant.
- **Application B (P0):** Signed repair pack allowlist for AI-agent OS tool execution.
- **Application C (P0):** Offline connectivity decision tree integrated into AI agent pipeline.

Applications D and E are recommended as follow-on filings within six months if search results are favorable.

Public disclosure

We have published source code and software releases on GitHub. Please advise on any impact on patentability and recommended filing dates. We will provide exact commit and release dates upon request.

Contact

Role	Name	Email	Phone
Primary contact			
Technical contact			
Authorized signatory			

We appreciate your prompt review and a proposal for engagement including estimated fees for provisional filing, prior-art search, and non-provisional preparation.

Sincerely,

[Name, Title]
[Company Legal Name]

Enclosures: Patent disclosure package (Documents 00–08, screenshots, optional PDF)

Invention Disclosure: Jonathan Autonomous AI Technician

Invention title (working): Local-First Autonomous IT Incident Orchestration with Execute-Then-Narrate AI Technician

Product: Thorpe Desktop

Agent name: Jonathan

Disclosure type: Utility patent — system and method

1. Field of the invention

Computer systems diagnostics, automated remediation, and AI-assisted technical support on end-user devices (desktops and laptops) across Windows, macOS, and Linux.

2. Background and problem solved

Traditional IT support chatbots:

- Recommend manual steps (open Settings, run a command, reboot)
- Depend on continuous cloud connectivity
- Do not verify whether suggested actions improved system health
- Conflate AI text generation with actual system changes
- Lack enterprise controls over which OS-level actions an AI may invoke

Jonathan solves this by treating the AI as a **narrator and planner** layered on top of a **deterministic local repair engine** that runs on the device. Repairs execute **before** the user sees the assistant's reply. The AI is constrained not to invent repairs—it may only describe repairs already executed and recorded.

3. Summary of the invention

Jonathan is implemented as a pipeline:

```
User message → Evidence collection → Security gate → Optional offline connectivity suite
→ Repair planning (LLM JSON or rules) → Repair-pack allowlist filter → Confirmation gate
→ Local OS repair execution → Post-mutating verification scan → Session persistence
→ Narrative generation (local template or cloud LLM) → Structured chat response
```

The user receives a single response containing: narrative text, list of repairs executed, pending repairs requiring approval, verification metrics, connectivity report (if applicable), agent plan, knowledge suggestions, and optional escalation case ID.

4. Detailed operation — user chat flow

4.1 Frontend (JonathanAssistant.tsx)

1. User enters a natural-language issue (e.g., "Wi-Fi not working", "computer is slow").
2. Frontend calls Tauri command chat_with_jonathan with:
 - message: user text
 - skill_level: beginner | advanced (affects explanation depth)
 - scan_context: optional JSON of last system scan
 - history: prior chat turns
 - confirmed_repairs: optional list of repair action IDs when user clicks **Approve**
3. Assistant message metadata is persisted and restored from metadata_json (repairs, plan, verification, connectivity).

4.2 Entry point — chat_with_jonathan (ai/mod.rs)

Order of operations (critical for patent characterization):

1. Parse optional scan context into SystemScanResult
2. Save user message to chat_history
3. Call orchestrate_incident() — **all repairs run here**
4. Convert IncidentResult to ChatExtras (repairs, pending, verification, etc.)
5. Resolve narrative source:
 - Enterprise AI runtime (source: "enterprise")
 - User cloud OpenAI-compatible API (source: "openai")
 - Local deterministic formatter (source: "local")
 - Fallback variants on policy/error
6. Enrich message with structured sections (case ID, pending repairs, verification, connectivity, plan)
7. Save assistant message with full metadata_json
8. Return ChatResponse

Key invariant: Cloud LLM failure does not prevent repairs from having already executed locally.

4.3 Orchestration — orchestrate_incident (agent/mod.rs)

Step	Action	Output
1	Generate session_id (UUID)	Session correlation
2	evidence::collect_system_evidence(scan)	Event log excerpt, service summary, network summary
3	Persist evidence artifacts (system_evidence, scan_snapshot)	Audit trail
4	Seed intel feed and repair packs	Planner tool availability
5	Security escalation check	If malware/credential keywords → create PSA case, no repairs

6	Connectivity pre-run (if network keywords)	ConnectivityReport, DB record, evidence artifact
7	<code>build_plan()</code> — enterprise LLM → user LLM → rules	AgentPlan
8	Extract <code>tool_id</code> from plan steps	Planned repair IDs
9	<code>filter_allowed_tools()</code> — intersect with repair-pack allowlist	Gated tool list
10	If connectivity already ran, remove duplicate <code>connectivity-suite</code>	Dedup
11	<code>plan_chat_repairs(planned, confirmed_repairs)</code>	<code>to_run</code> + <code>pending_confirmation</code>
12	<code>perform_repairs_with_failures(db, to_run)</code>	<code>Vec<RepairResult></code>
13	If any mutating repair succeeded → <code>quick_system_scan()</code>	RepairVerification
14	<code>gather_kb()</code> — knowledge FTS + intel search	<code>KbSuggestion[]</code>
15	<code>persist_session()</code> — plan, evidence, confidence	AgentSessionRecord + PSA webhook
16	If <code>plan.confidence < 0.55</code> → low-confidence escalation case	Internal case (optional)

Returns `IncidentResult` to narrative layer.

5. Repair planning

5.1 Rule-based planner (`repairs/planner.rs`)

Maps user message keywords and scan signals to repair tool IDs:

- Network terms → `connectivity-suite`, `dns-flush`
- Performance terms → `high-resource-id`, `startup-review`, `temp-cleanup`, `restart-recommend`
- Storage terms → `disk-analysis`, `temp-cleanup`
- Scan issue categories (storage, performance, network) → corresponding tools
- High memory (>85%) → `high-resource-id`

5.2 LLM planner (`agent/planner.rs`)

When enterprise or user cloud runtime is available:

- Builds `PlannerContext` with message, scan JSON, evidence JSON, KB excerpts (3), intel excerpts (3), available tools
- Calls OpenAI-compatible `/chat/completions` with `response_format: json_object`, temperature 0.2

- Parses JSON into AgentPlan: hypotheses, confidence, steps[], citations, escalate_if
- Each step: tool_id, reason, risk, requires_approval
- Filters steps to tools present in available_tools

Falls back to plan_with_rules if LLM unavailable or returns empty steps.

5.3 Repair action taxonomy (repairs/mod.rs)

Kind		Behavior	Examples
diagnostic	Auto-execute without confirmation	connectivity-suite, disk-analysis, high-resource-id	
mutating	Requires user confirmation	dns-flush, temp-cleanup, print-spooler-restart	
advisory	Informational only	startup-review, update-check, restart-recommend	

plan_chat_repairs splits planned IDs into immediate execution vs. pending approval based on requires_confirmation and confirmed_repairs list.

6. Repair execution and verification

6.1 Execution

perform_repair(db, action_id):

1. Looks up RepairAction from catalog
2. Runs platform-specific command (PowerShell, shell, etc.) via execute_action
3. Records result in repair_history SQLite table
4. Returns RepairResult with success, message, details, action_kind

6.2 Post-mutating verification

When any_mutating_success(repairs_executed):

```
rust
RepairVerification {
    health_before,    // from scan context or 0
    health_after,     // from quick_system_scan()
    issues_before,
    issues_after,
    improved: health_after > health_before || issues_after < issues_before
}
```

Surfaced in UI and appended to both local and cloud narratives.

7. Offline connectivity diagnostics

Trigger: `is_connectivity_issue(message)` — keyword detection in user message.

Suite (`run_connectivity_suite`): Layered checks without internet:

1. Network adapter status (OS-specific)
2. Default gateway reachability (ping)
3. DNS resolution (example.com)
4. Internet reachability (ping 1.1.1.1, 8.8.8.8)

Playbook (`evaluate_playbook`): Local decision tree produces:

- `overall_status`: offline | degraded | healthy
- `recommended_actions`: repair tool IDs
- `playbook_summary`: human-readable assessment
- `offline_capable`: true

Agent integration:

- Full report saved to `connectivity_diagnostics` table
- Evidence artifact `offline_connectivity_suite` linked to session
- `connectivity-suite` removed from repair plan if suite already ran
- Report embedded in `ChatResponse.connectivity_report`

8. Narrative modes (autonomous vs cloud)

8.1 Local autonomous (`format_technician_response`)

Deterministic markdown builder using:

- User first name personalization
- Repairs executed (grouped by diagnostic / mutating / advisory)
- Pending repairs list
- Verification block
- Connectivity summary
- Agent plan steps
- KB suggestions

No network required.

8.2 Cloud narration (`call_openai_compatible`)

System prompt explicitly states repairs **already ran**. LLM must:

- Describe repairs in past tense
- Not invent actions not in `repairs_executed`

- Never request credentials
- Personalize by first name

Temperature 0.7 for conversational tone.

8.3 Enterprise AI layer (`enterprise_ai/mod.rs`)

`resolve_runtime("jonathan")` gates cloud use on:

- License feature `enterprise_ai_console`
- Org policy: `cloud_ai_enabled`, monthly budget, token limits
- Agent enabled + user role in `allowed_roles`
- Provider enabled + API key configured

Usage logged to audit table.

9. Repair pack security model

Purpose: Control which OS-level tools Jonathan may invoke.

1. **Built-in packs** (`thorpe-core`, `thorpe-network`, `thorpe-performance`) ship with app
2. **Custom packs** installed via `install_repair_pack(manifest_json)`
3. Manifest signed with **Ed25519** (or legacy HMAC) — `pack_signing.rs`
4. `allowed_tool_ids(db)` = union of enabled pack tools
5. `filter_allowed_tools` intersects planner output with allowlist before execution

CLI tools: `thorpe-pack-sign`, `thorpe-license-key`

10. Proactive watchdog integration

Background loop (`watchdog/mod.rs`):

- Interval: configurable (default 15 min, minimum 5)
- Runs `quick_system_scan()`
- **Per-metric alerts:** CPU >80%, memory >85%, disk >90%, health below threshold
- Dedupes unacknowledged alerts per `event_type` within interval
- Optional `auto_plan`: pre-computes `AgentPlan` via `plan_with_rules`
- Emits Tauri event `watchdog-alert` to UI

Handoff to Jonathan:

1. User clicks "Send to Jonathan" (Settings or notification)
2. `WatchdogHandoff` stored in app state; navigate to `/jonathan`
3. Synthetic user prompt built: Watchdog alert (CPU spike): ... Health score is 68/100. Please investigate...

4. Full chat_with_jonathan / orchestrate_incident pipeline runs
5. Pre-loaded plan merged into response if API plan absent
6. Event acknowledged on success

11. Intelligence console and RAG

Enterprise-gated (intelligence_console license feature):

- Threat intel feed sync (sync_intel_feed)
- Org playbooks (upsert_org_playbook) — content indexed into knowledge_fts on save
- Repair pack management
- Agent session audit and PDF export

Planner RAG: search_knowledge_for_message (FTS) + search_intel_for_message feed excerpts into PlannerContext. Playbooks surface through same FTS path as KB articles.

12. Security and escalation

Trigger	Behavior
Malware/ransomware/virus keywords	No repairs; SupportCase created; PSA case.escalated
Password/credential keywords	Same
plan.confidence < 0.55	Low-confidence escalation case (internal)
Mutating repairs	User approval required in chat or Repair Center

Jonathan system prompt forbids credential requests.

13. Data persistence

Table	Contents
chat_history	Messages + metadata_json (full incident state)
agent_sessions	Plan JSON, evidence JSON, confidence, status
evidence_artifacts	Per-session evidence blobs
repair_history	Executed repair log
connectivity_diagnostics	Offline suite results

watchdog_events	Proactive alerts + plan JSON + issues JSON
knowledge_fts	KB + org playbooks (FTS5)

PSA webhooks: agent.session.completed, case.escalated, case.created with HMAC signature.

14. Novel technical combinations (for patentability discussion)

Attorney to evaluate against prior art:

1. **Execute-then-narrate:** Local repair engine completes before any LLM generates user-visible text; LLM constrained to factual repair log.
2. **Dual-LLM architecture:** Separate JSON planner and conversational narrator with independent fallbacks.
3. **Signed repair-pack allowlist:** Cryptographic gate on autonomous OS tool execution.
4. **Offline connectivity playbook fused into agent path** with duplicate diagnostic suppression.
5. **Post-mutating verification micro-scan** feeding structured chat metadata.
6. **Watchdog-to-agent handoff** reusing full incident pipeline with pre-computed plan.
7. **Confirmation-threaded chat orchestration:** Same orchestrate_incident handles initial and approval messages via confirmed_repairs.
8. **Tiered AI governance:** Free local autonomous repairs + optional user cloud + enterprise role/budget policy on same engine.

15. Reduction to practice

- **Status:** Implemented and tested in Thorpe Desktop v1.1.0 codebase
- **Platforms:** Windows (NSIS/MSI), macOS (DMG), Linux (AppImage/deb) via Tauri 2
- **Tests:** 70+ frontend tests, 40 Rust unit tests including watchdog, connectivity, repair planning
- **Public disclosure risk:** GitHub repository and releases may affect filing timeline — attorney to advise on bar dates

16. Figures referenced

See [03-SYSTEM-ARCHITECTURE-AND-FLOWS.md](./03-SYSTEM-ARCHITECTURE-AND-FLOWS.md) for:

- Figure 1: System architecture block diagram
- Figure 2: Chat turn sequence diagram
- Figure 3: Orchestration flowchart
- Figure 4: Watchdog handoff sequence
- Figure 5: Repair pack gating flow

17. Glossary

Term	Definition
Jonathan	Embedded AI technician agent in Thorpe Desktop
Repair action / tool	Atomic diagnostic or mutating OS operation with ID (e.g., <code>dns - flush</code>)
Agent plan	Structured JSON plan with hypotheses, steps, confidence
Incident	One user message processed through <code>orchestrate_incident</code>
Mutating repair	OS-changing action requiring explicit user confirmation
Repair pack	Signed manifest defining allowed repair tools
Watchdog	Background proactive health monitor

Patent Candidate Inventory — Thorpe Desktop / Jonathan

Purpose: Request for patent counsel to evaluate, prioritize, and file applications to protect the Thorpe Desktop concept and Jonathan autonomous technician architecture.

Recommended jurisdiction: United States (USPTO) utility patents first; consider PCT within 12 months for international coverage.

Filing strategy note: Consider an **early provisional** on Patent A to establish priority date, followed by non-provisional with continuations for B–F within 12 months.

Priority tier summary

Tier	Patents	Rationale
P0 — File first	A, B, C	Core differentiation; highest competitive value
P1 — File within 6 months	D, E	Strong supplementary protection
P2 — Evaluate after search	F, G, H	Narrower or design; depends on prior art
P3 — Trade secret / copyright	UI flows, prompt text	May be better protected non-patent

Patent A — Core autonomous incident orchestration (BROAD)

Working title: System and Method for Local-First Autonomous IT Incident Resolution with Execute-Then-Narrate AI Assistant

Type: Utility — system + method claims

Abstract (draft): A computer-implemented system on an end-user device receives a natural-language support request, collects local system evidence, plans a set of repair actions using rule-based and/or LLM-based planning, filters planned actions against a signed allowlist, executes non-mutating diagnostics automatically and defers mutating repairs pending user confirmation, performs a follow-up health scan after successful mutating repairs, and only thereafter generates a user-facing narrative constrained to describe repairs already executed.

Key elements to claim:

- Execute-then-narrate ordering (repairs before LLM narrative)
- Dual planner fallback (LLM JSON plan → rule plan)
- Confirmation gate for mutating vs. diagnostic actions
- Post-mutating verification scan with structured improvement metrics
- Structured chat response with repairs, pending, verification, plan metadata
- Local-first operation without cloud dependency

Code anchors: `orchestrate_incident`, `chat_with_jonathan`, `plan_chat_repairs`, `format_technician_response`

Estimated claim count: 1 independent system + 1 independent method + 15–25 dependent

Priority: P0 — Provisional recommended immediately

Patent B — Signed repair pack gated autonomous execution

Working title: Cryptographically Signed Repair Pack Allowlist for AI-Agent OS Tool Execution

Type: Utility

Abstract (draft): A method for controlling which operating-system-level repair tools an autonomous AI agent may invoke, comprising installing repair packs with Ed25519-signed manifests, maintaining an allowlist union of enabled pack tools, and intersecting AI planner output with the allowlist before execution.

Key elements:

- Repair pack manifest structure (tools, version, `maps_to_action`)
- Ed25519 signature verification on install
- Runtime intersection of planner tool IDs with `allowed_tool_ids`
- Built-in vs. custom pack distinction; built-in packs non-overwritable

Code anchors: `pack_signing.rs`, `packs.rs`, `filter_allowed_tools`, `install_repair_pack`

Priority: P0

Patent C — Offline layered connectivity diagnostics integrated into AI agent pipeline

Working title: Offline Network Connectivity Decision Tree with Agent Incident Fusion and Diagnostic Deduplication

Type: Utility

Abstract (draft): Upon detecting network-related user intent, a local agent executes a layered connectivity suite (adapter, gateway, DNS, internet reachability) without cloud connectivity, evaluates a local playbook to produce recommended repair actions and summary, persists results as session evidence, suppresses duplicate connectivity diagnostics in a subsequent repair plan, and embeds the report in an AI chat response.

Key elements:

- Keyword-triggered pre-run before planning
- Layered checks with pass/fail/warn status
- Local playbook decision tree → `recommended_actions`
- `offline_capable: true` semantic
- Dedup: remove `connectivity-suite` from plan if suite already executed
- Persistence to diagnostics table + evidence artifacts

Code anchors: `connectivity/mod.rs`, `orchestrate_incident` connectivity block

Priority: P0

Patent D — Proactive watchdog to autonomous agent handoff

Working title: Background Health Monitor with Pre-Computed Repair Plan Handoff to Conversational AI Agent

Type: Utility

Abstract (draft): A background monitoring service periodically scans system health, generates per-metric alerts (CPU, memory, disk, composite health), optionally pre-computes a repair plan using the same rule planner as the conversational agent, notifies the user, and upon user initiation injects a synthetic support request into the same incident orchestration pipeline used for interactive chat, merging any pre-computed plan with agent-generated results.

Key elements:

- Per-metric alert types with deduplication per event type
- Pre-computed `plan_json` on watchdog events
- Tauri event → UI notification → "Send to Jonathan"
- Shared `orchestrate_incident` pipeline for watchdog and chat

- Acknowledgment on successful handoff

Code anchors: watchdog/mod.rs, watchdog.ts, JonathanAssistant.tsx handoff effect

Priority: P1

Patent E — Dual-LLM planner/narrator with independent fallbacks

Working title: Separated Planning and Narration Language Models for Autonomous IT Support with Rule-Based Fallback

Type: Utility

Abstract (draft): An IT support system employs a first LLM call producing structured JSON repair plans filtered to an allowlist, and a second LLM call generating user narrative constrained to repairs already executed, each independently fallible to rule-based planning and template-based narration respectively.

Key elements:

- Planner: JSON AgentPlan, temperature 0.2, tool filtering
- Narrator: conversational, temperature 0.7, anti-hallucination prompt
- Enterprise vs. user vs. local runtime resolution
- Independent fallback paths

Code anchors: agent/planner.rs, ai/mod.rs, build_plan, resolve_user_cloud_response

Priority: P1

Patent F — Security keyword circuit breaker for autonomous repair agents

Working title: Autonomous IT Agent Security Escalation Gate Blocking OS Repairs on Credential and Malware Indicators

Type: Utility (narrower)

Key elements:

- Keyword detection for malware, ransomware, virus, password, credential
- Bypass repair engine entirely
- Automatic support case creation + PSA webhook
- Return structured escalation without mutating repairs

Code anchors: security_escalation, is_security_escalation in agent/mod.rs

Priority: P2

Patent G — Org playbook full-text search fusion for AI planner context

Working title: Organizational Playbook Indexing into Unified Full-Text Search for AI Repair Planning

Type: Utility (narrower)

Key elements:

- Org playbooks written to FTS5 virtual table on save
- Same search path as knowledge base for planner excerpts
- Intel feed search in parallel

Code anchors: db/mod.rs rebuild_knowledge_fts_from_playbooks, build_planner_context

Priority: P2

Patent H — Tiered license-governed AI runtime (Enterprise)

Working title: Multi-Tier License Feature Gating for Autonomous Local Repairs and Enterprise AI Policy Layer

Type: Utility (business method + system — evaluate Alice risk)

Key elements:

- Free tier: local autonomous repairs without cloud
- Enterprise: role-based provider access, budget enforcement, audit log
- Same repair engine across tiers; cloud gated separately from chat

Code anchors: licensing/mod.rs, enterprise_ai/mod.rs

Priority: P2 — may be difficult to claim; consider trade secret for pricing tiers

Design patents (optional)

ID	Subject	Notes
D1	Jonathan chat UI with repair approval cards	Distinctive approval-needed amber banner + Approve button
D2	Health score ring + dashboard layout	If visually distinctive
D3	Connectivity report inline in chat	Layered check list presentation

Design patents are **lower priority** than utility patents for core technology.

Defensive publications (alternative to filing)

If budget-constrained, consider publishing narrowly on:

- Specific keyword lists for plan_repairs
- Exact health score penalty formula

This prevents others from patenting identical specifics while keeping broad claims for Patents A–C.

Estimated filing budget (for client planning — attorney to quote)

Item	Typical range (US)
Provisional (Patent A)	\$3,000–\$8,000 + USPTO fees
Non-provisional (A)	\$12,000–\$25,000 + fees
Each additional non-provisional (B–E)	\$8,000–\$18,000 each
Prior art search (comprehensive)	\$2,000–\$5,000
PCT national phase (per country)	Varies

Recommended filing sequence

Month 0: Provisional – Patent A (core orchestration)

Month 0: Provisional – Patent B (repair packs) OR combine with A if budget limited

Month 1: Prior art search results → refine claims

Month 2: Provisional – Patent C (connectivity) if not combined

Month 6: Non-provisional A + continuations/in divisionals for B, C

Month 6: File D, E if search favorable

Month 12: PCT decision

Request to patent attorney

Please provide:

1. Patentability opinion on Patents A–E against your search
2. Recommendation on combining vs. separating applications
3. Alice/abstract idea analysis for software method claims
4. Inventorship review (Document 06)
5. Assignment to operating company

6. Filing timeline given any public GitHub disclosure dates
7. Trademark clearance for "Jonathan" and "Thorpe" in Class 9/42

System Architecture and Flows — Thorpe Desktop / Jonathan

For: Patent drawings and specification figures

Format: Mermaid diagrams (attorney may convert to USPTO-compliant line drawings)

Figure 1 — System architecture (block diagram)

```
mermaid
flowchart TB
    subgraph UI["Presentation Layer (React / Tauri WebView)"]
        JA[Jonathan Assistant Chat]
        RC[Repair Center]
        SS[System Scanner]
        WD_UI[Watchdog Settings / Alerts]
        IC[Intelligence Console]
    end

    subgraph API["Tauri Command API"]
        CWJ[chat_with_jonathan]
        ER[execute_repair]
        RSS[run_system_scan]
        GWS[get_watchdog_status]
        RCD[run_connectivity_diagnostics]
    end

    subgraph ORCH["Incident Orchestration (Rust)"]
        OI[orchestrate_incident]
        BP[build_plan]
        PWL[plan_with_llm]
        PWR[plan_with_rules]
        FAT[filter_allowed_tools]
        PCR[plan_chat_repairs]
        PRF[perform_repairs_with_failures]
        VS[quick_system_scan_verification]
    end

    subgraph AI["Narrative Layer"]
        LOC[format_technician_response]
        CLD[call_openai_compatible]
        ENT[enterprise_ai_resolve_runtime]
    end
```



```

subgraph DATA["Local Persistence (SQLite)"]
    CH[chat_history]
    AS[agent_sessions]
    EV[evidence_artifacts]
    RH[repair_history]
    CD[connectivity_diagnostics]
    WE[watchdog_events]
    FTS[knowledge_fts]
end

```

```

subgraph OS["Operating System"]
    CMD[Shell / PowerShell Commands]
    NET[Network Stack / Ping / DNS]
end

```

```

JA --> CWJ
WD_UI -->|Send to Jonathan| JA
CWJ --> OI
OI --> BP
BP --> PWL
BP --> PWR
OI --> FAT
OI --> PCR
PCR --> PRF
PRF --> CMD
OI --> VS
VS --> NET
CWJ --> LOC
CWJ --> CLD
CLD --> ENT
OI --> CH
OI --> AS
OI --> EV
PRF --> RH
OI --> CD

```

Figure 2 — Chat turn sequence (method steps)

```

mermaid
sequenceDiagram
    participant U as User
    participant UI as Jonathan UI
    participant Chat as chat_with_jonathan
    participant Orch as orchestrate_incident

```

```

    participant Plan as build_plan
    participant Repair as Repair Engine
    participant Scan as quick_system_scan
    participant Narr as Narrative Layer

U->>UI: Natural language issue
UI->>Chat: ChatRequest (message, scan, history, confirmed_repairs?)
Chat->>Orch: orchestrate_incident()
Orch->>Orch: collect evidence, security check
opt Network keywords
    Orch->>Orch: run_connectivity_suite()
end
Orch->>Plan: build_plan (LLM or rules)
Plan-->>Orch: AgentPlan
Orch->>Orch: filter_allowed_tools (repair packs)
Orch->>Repair: plan_chat_repairs + perform_repairs
Repair-->>Orch: RepairResult[]
opt Mutating success
    Orch->>Scan: quick_system_scan()
    Scan-->>Orch: RepairVerification
end
Orch-->>Chat: IncidentResult
alt Cloud available
    Chat->>Narr: LLM narrate (repairs already done)
else Autonomous
    Chat->>Narr: format_technician_response
end
Narr-->>Chat: message text
Chat-->>UI: ChatResponse (repairs, pending, verification, plan)
UI-->>U: Display + optional Approve button

```

Figure 3 — Orchestration decision flowchart

```

mermaid
flowchart TD
    START([User message received]) --> EVID[Collect system evidence]
    EVID --> SEC{Security keywords?}
    SEC -->|Yes| CASE[Create escalation case<br/>No repairs]
    SEC -->|No| NET{Connectivity keywords?}
    NET -->|Yes| SUITE[Run offline connectivity suite<br/>Save evidence]
    NET -->|No| PLAN
    SUITE --> DEDUP[Mark connectivity-suite for dedup]
    DEDUP --> PLAN[build_plan: LLM or rules]
    PLAN --> FILTER[filter_allowed_tools]
    FILTER --> DEDUP2{Connectivity already ran?}
    DEDUP2 -->|Yes| REM[Remove connectivity-suite from plan]
    DEDUP2 -->|No| CONF
    REM --> CONF[plan_chat_repairs + confirmed_repairs]
    CONF --> EXEC[Execute to_run repairs]

```

```

EXEC --> MUT{Mutating success?}
MUT -->|Yes| VERIFY[quick_system_scan verification]
MUT -->|No| PERSIST
VERIFY --> PERSIST[Persist session + KB suggestions]
PERSIST --> LOW{confidence < 0.55?}
LOW -->|Yes| ESC[Low-confidence case]
LOW -->|No| RETURN([IncidentResult])
CASE --> RETURN
ESC --> RETURN

```

Figure 4 — Watchdog to Jonathan handoff

```

mermaid
sequenceDiagram
    participant WD as Watchdog Loop
    participant DB as SQLite
    participant UI as App Shell
    participant NC as Notification Center
    participant JA as Jonathan Assistant
    participant Chat as chat_with_jonathan

    WD->>WD: quick_system_scan()
    WD->>WD: detect_metric_alerts (CPU, memory, disk, health)
    WD->>WD: plan_with_rules (optional auto_plan)
    WD->>DB: save_watchdog_event
    WD->>UI: emit watchdog-alert
    UI->>NC: Notification + Send to Jonathan action
    NC->>JA: setWatchdogHandoff + navigate
    JA->>JA: buildWatchdogJonathanPrompt()
    JA->>Chat: chatWithJonathan (synthetic message)
    Chat->>Chat: orchestrate_incident (full pipeline)
    Chat->>JA: ChatResponse + merged plan
    JA->>DB: acknowledge_watchdog_event

```

Figure 5 — Repair pack gating

```

mermaid
flowchart LR
    MANIFEST[Signed Repair Pack Manifest] -->
    INSTALL[install_repair_pack<br/>Ed25519 verify]
    INSTALL --> DB[(repair_packs table)]
    DB --> ALLOW[allowed_tool_ids union]
    PLAN[AgentPlan steps] --> IDS[tool_id list]
    IDS --> INTERSECT1{filter_allowed_tools}
    ALLOW --> INTERSECT

```

```
INTERSECT --> SAFE[Filtered tool IDs]
SAFE --> EXEC[perform_repairs]
```

Key data structures (for specification)

ChatRequest

```
message: string
skill_level: string
scan_context: optional SystemScanResult JSON
history: ChatHistoryItem[]
confirmed_repairs: optional string[] (repair action IDs)
```

AgentPlan

```
hypotheses: string[]
confidence: float (0.0–1.0)
steps: AgentPlanStep[]
citations: string[]
escalate_if: string[]
```

AgentPlanStep

```
tool_id: string
reason: string
risk: string
requires_approval: boolean
```

RepairResult

```
success: boolean
message: string
action_id: string
action_name: string
action_kind: diagnostic | mutating | advisory
record_id: string
```

RepairVerification

```
health_before: int
health_after: int
issues_before: int
issues_after: int
improved: boolean
```

ConnectivityReport

```
checks: ConnectivityCheck[]
overall_status: offline | degraded | healthy
recommended_actions: string[]
playbook_summary: string
offline_capable: true
```

WatchdogEvent

```
id: string
event_type: health_threshold | high_cpu | high_memory | low_disk:{mount}
health_score: int
message: string
plan_json: optional AgentPlan JSON
issues_json: optional ScanIssue[] JSON
acknowledged: boolean
```

Platform deployment

Component	Technology
Desktop shell	Tauri 2 (Rust + WebView)
Frontend	React 18, TypeScript
Backend	Rust (scanner, repairs, agent, AI)
Database	SQLite with FTS5
Cloud AI	OpenAI-compatible HTTPS API (optional)
Targets	Windows 10/11, macOS Apple Silicon, Linux

Suggested patent figure captions

- 1. **FIG. 1** — Block diagram of Thorpe Desktop autonomous IT support system showing presentation layer, orchestration engine, narrative layer, local persistence, and operating system interface.
- 2. **FIG. 2** — Sequence diagram illustrating execute-then-narrate ordering wherein repair execution precedes AI narrative generation.
- 3. **FIG. 3** — Flowchart of incident orchestration including security gate, connectivity pre-run, planning, allowlist filtering, confirmation gate, execution, and verification.

4. **FIG. 4** — Sequence diagram of proactive watchdog alert handoff into conversational agent pipeline.

5. **FIG. 5** — Flowchart of cryptographically signed repair pack allowlist intersection with AI planner output.

Claim Seeds for Attorney Drafting

Disclaimer: These are **seed claims** for patent counsel to refine, broaden, narrow, and prosecute. They are not filed claims and have not been reviewed for patentability, prior art, or formal patent law requirements (35 U.S.C. § 101, § 102, § 103, § 112).

Patent A — Core orchestration

Independent claim 1 (system — seed)

A computer system comprising at least one processor and memory storing instructions that, when executed, cause the system to:

- (a) receive, at a local client device, a natural-language user message describing a computing device problem;
- (b) collect local system evidence from the client device, the local system evidence including at least one of operating system state, resource utilization, or network state;
- (c) generate a repair plan comprising a plurality of repair action identifiers selected from a catalog of repair actions, wherein generating the repair plan comprises applying at least one of a rule-based planner or a language model that outputs a structured plan;
- (d) filter the repair plan by intersecting the repair action identifiers with an allowlist of permitted repair actions;
- (e) automatically execute, on the client device without user interaction, repair actions classified as non-mutating diagnostics;
- (f) defer execution of repair actions classified as mutating until receipt of user confirmation;
- (g) after successful execution of at least one mutating repair action, perform a follow-up system health scan and compute a verification metric comparing health before and after execution;
- (h) generate a user-facing narrative describing repairs executed in step (e) and optionally step (f), wherein the narrative is constrained to repairs having recorded execution results; and
- (i) present, in a single user interface response, the narrative, the recorded execution results, any deferred repair actions, and the verification metric.

Independent claim 2 (method — seed)

A computer-implemented method performed on a client device, comprising: [mirror steps (a)–(i) from claim 1 in method form].

Dependent claims (seeds — select and refine)

3. The system of claim 1, wherein generating the repair plan comprises calling a first language model to produce a JSON-structured plan and, upon failure of the first language model, falling back to a rule-based planner mapping keywords in the user message to repair action identifiers.
4. The system of claim 1, wherein generating the user-facing narrative comprises calling a second language model separate from the first language model used to generate the repair plan.
5. The system of claim 1, wherein the allowlist is derived from cryptographically signed repair pack manifests installed on the client device.
6. The system of claim 1, further comprising, prior to step (c), detecting security-sensitive keywords in the user message and, upon detection, creating a support case record and skipping steps (e) and (f).
7. The system of claim 1, further comprising, prior to step (c), upon detecting network-related keywords, executing an offline connectivity diagnostic suite without cloud network access and storing results as session evidence.
8. The system of claim 7, further comprising removing a connectivity diagnostic repair action identifier from the repair plan when the offline connectivity diagnostic suite was already executed for the session.
9. The system of claim 1, wherein the user-facing narrative is generated by a local template formatter when a cloud language model is unavailable.
10. The system of claim 1, wherein the single user interface response includes structured metadata persisted to enable restoration of repair results and agent plan upon session reload.

Patent B — Repair packs

Independent claim (seed)

A method for controlling autonomous operating-system repair tool execution on a client device, comprising:

receiving a repair pack manifest specifying a plurality of repair tools and a digital signature;

verifying the digital signature using a public key embedded in a client application;

upon successful verification, storing the repair pack and enabling the repair tools in an allowlist;

receiving a planned set of repair tool identifiers from an AI planning module; and

executing only those planned repair tool identifiers present in the allowlist.

Dependent claims (seeds)

- wherein the digital signature is an Ed25519 signature over canonical JSON of the manifest
- wherein built-in repair packs are non-overwritable by custom packs
- wherein each repair tool maps to a canonical repair action identifier in a global catalog

Patent C — Offline connectivity

Independent claim (seed)

A computer-implemented method for offline network diagnostics in an AI-assisted support application, comprising:

detecting, from a user message, network connectivity intent;

executing, without requiring internet connectivity to a cloud AI service, a layered connectivity suite comprising adapter status, gateway reachability, domain name resolution, and internet endpoint reachability checks;

evaluating check results with a local decision tree to produce recommended repair action identifiers and a playbook summary;

persisting the connectivity results as evidence linked to an agent session;

generating a repair plan for the user message;

removing a duplicate connectivity diagnostic identifier from the repair plan when the layered connectivity suite was executed; and

including the connectivity results in a chat response to the user.

Patent D — Watchdog handoff

Independent claim (seed)

A method for proactive system monitoring integrated with a conversational repair agent, comprising:

periodically scanning, by a background monitor on a client device, system health metrics including at least CPU utilization, memory utilization, and storage utilization;

generating an alert when a metric exceeds a corresponding threshold;

optionally pre-computing a repair plan for the alert using a rule-based planner;

notifying a user of the alert;

in response to user selection of a handoff action, injecting a synthetic support request into a conversational agent pipeline that executes the same incident orchestration process used for interactive chat, including local

repair execution and narrative generation; and

acknowledging the alert upon successful completion of the incident orchestration.

Dependent claims (seeds)

- wherein alerts are deduplicated per alert type within a configurable time window
- wherein the pre-computed repair plan is merged into an agent response when a cloud planner is unavailable

Patent E — Dual LLM

Independent claim (seed)

A system comprising a client device configured to:

invoke a first language model to produce a structured repair plan in machine-readable format;

filter plan steps to an allowlist of repair tools;

execute permitted repair tools on the client device;

invoke a second language model, independent of the first language model, to generate a natural-language summary constrained to describe only repair tools already executed; and

upon failure of either language model, fall back to a respective non-LLM alternative selected from rule-based planning or template-based narration.

Patent F — Security gate

Independent claim (seed)

A method comprising: receiving a user message at an autonomous IT repair agent; classifying the message as a security escalation based on keyword detection for at least one of malware indicators or credential-related terms; in response to classification as security escalation, creating a support case, transmitting a webhook notification to an external professional services automation system, and inhibiting execution of automated repair actions; and returning an escalation identifier to a user interface without performing mutating repairs.

Means-plus-function note

Attorney to decide whether means-plus-function language is appropriate for:

- "module for orchestrating incidents"

- "module for verifying repair packs"

Specific structure is available in Rust modules (agent/mod.rs, repairs/pack_signing.rs).

Claim chart template (for prosecution)

Claim element	Code implementation	Figure
Receive user message	chat_with_jonathan	FIG. 2
Collect evidence	evidence::collect_system_evidence	FIG. 3
Generate repair plan	build_plan	FIG. 3
Filter allowlist	filter_allowed_tools	FIG. 5
Execute repairs	perform_repairs_with_failures	FIG. 2
Verification scan	quick_system_scan	FIG. 3
Generate narrative	format_technician_response / call_openai_compatible	FIG. 2

Prior Art and Differentiation Notes

Purpose: Guide patent counsel's prior-art search and prosecution strategy. This is not a completed prior-art search.

1. Categories of known art (to search)

1.1 Conversational IT support chatbots

Examples to search: Microsoft Support virtual agent, Apple Support app chat, generic ChatGPT IT troubleshooting, Intercom/Zendesk AI bots.

Typical limitations:

- Recommend manual steps; do not execute OS commands locally
- No structured repair result object returned with chat
- No post-action verification scan
- Cloud-dependent

Thorpe/Jonathan differentiation:

- Execute-then-narrate: repairs complete before LLM response

- Structured ChatResponse with repairs_executed, pending_repairs, verification
- Works fully offline for diagnostics and local narrative

1.2 Remote monitoring and RMM (MSP tools)

Examples to search: ConnectWise Automate, Datto RMM, NinjaOne, Microsoft Intune remediations.

Typical limitations:

- Agent managed by MSP, not end-user conversational
- Scripted remediations, not NL-driven planning
- No consumer-facing AI persona
- Requires central management server

Differentiation:

- User-initiated natural language on local device
- Jonathan persona with chat approval UX
- Optional PSA webhook integration (outbound), not full RMM stack

1.3 OS built-in troubleshooters

Examples: Windows Troubleshooter, macOS Diagnostics, Linux network-manager troubleshoot.

Typical limitations:

- Fixed decision trees per symptom category
- No LLM planning or narration
- No chat interface with history
- No signed extensible repair packs

Differentiation:

- LLM + rules hybrid planner
- Signed OTA repair packs
- Unified incident session with evidence artifacts

1.4 LLM tool-use / function-calling agents

Examples: OpenAI function calling, LangChain agents, AutoGPT, Microsoft Copilot actions.

Typical limitations:

- LLM decides and triggers tools in cloud loop
- Hallucinated tool invocations possible
- Narrative and execution interleaved, not execute-then-narrate
- Often cloud-only

Differentiation:

- Deterministic Rust repair engine owns execution order

- LLM planner output filtered by signed allowlist
- Narrator constrained by system prompt to executed repairs only
- Dual LLM with separate fallbacks

1.5 Endpoint security and repair utilities

Examples: Malwarebytes, CCleaner, built-in disk cleanup, DNS flush utilities.

Typical limitations:

- Single-purpose tools, not orchestrated incidents
- No AI chat layer
- No verification loop in chat context

Differentiation:

- Orchestrated multi-tool incidents from one message
- Health score verification after mutating repairs

1.6 Network diagnostic apps

Examples: ping/traceroute GUIs, Wi-Fi analyzer apps.

Differentiation:

- Layered offline playbook fused into AI agent pipeline
- Dedup against repair plan
- Embedded in chat metadata

1.7 Proactive PC health monitors

Examples: HWMonitor alerts, Windows Performance Monitor alerts, third-party RAM/disk alerts.

Differentiation:

- Handoff into same orchestration as interactive Jonathan chat
- Pre-computed AgentPlan on alert
- Per-metric typed alerts with deduplication

2. Academic / patent literature search terms

Use these in USPTO, Google Patents, Espacenet:

```
autonomous IT support agent local repair execution
AI chatbot system remediation desktop
execute then explain language model
signed manifest allowed tool list AI agent
offline network diagnostic decision tree
```

```
post repair verification health scan
conversational agent mutating action confirmation
MSP webhook AI session audit
dual language model planner narrator
proactive monitor handoff conversational agent
```

USPC/CPC classes to consider:

- G06F 11/07 (monitoring, testing, fault detection)
- G06F 11/34 (performance evaluation)
- G06F 9/455 (virtual machines — if claiming isolation)
- G06N 3/08 (learning methods — LLM aspects, Alice risk)
- G06F 21/55 (security — repair pack signing)
- H04L 41/08 (network fault management)
- G06F 3/048 (GUI — design patents)

3. Alice / abstract idea considerations (US)

Potential examiner rejection: Claims directed to "organizing human activity" or "mental processes" via generic computer implementation.

Proposed technical improvements to emphasize:

1. **Specific improvement to computer functionality** — automated OS-level repair execution with verification scan improves device operation, not merely presenting information
2. **Signed allowlist gate** — concrete security mechanism, not generic AI
3. **Offline connectivity suite** — specific layered network tests without cloud
4. **Execute-then-narrate ordering** — reduces hallucinated repairs, technical reliability improvement
5. **Post-mutating micro-scan** — measurable health metric change

Attorney: Consider emphasizing hardware/device-specific steps (ping, DNS resolution, shell commands) in independent claims.

4. Public disclosure inventory (bar date risk)

Disclosure	Risk	Notes
GitHub repository (Thorpe-Workforce)	High	Attorney to confirm earliest public commit dates
GitHub Releases (v1.0.8, etc.)	High	Published installers
Marketing website	Medium	If any
This patent package	Low	Internal to counsel if not published

Customer pilots	Medium	NDA may protect
-----------------	--------	-----------------

Action: File provisional **before** additional public marketing if not already barred.

5. Open source and third-party components

Component	License	Patent note
Tauri	Apache/MIT	No conflict
React	MIT	No conflict
ed25519-dalek	BSD	No conflict
OpenAI API	Commercial ToS	API use ≠ patent on API
SQLite	Public domain	No conflict

Thorpe-specific orchestration logic is original Rust/TypeScript in repository.

6. Competitive landscape watch list

Monitor patent filings from:

- Microsoft (Windows Copilot, Intune)
- Apple (Apple Intelligence, diagnostics)
- Google (Chrome OS diagnostics)
- CrowdStrike / SentinelOne (endpoint, different domain)
- NinjaOne, ConnectWise (RMM)

7. Differentiation summary table (for IDS / prosecution)

Feature	Prior art typical	Thorpe Jonathan
Repair execution	Manual or MSP remote	Local automatic before chat reply
LLM role	Planner + executor intertwined	Planner and/or narrator only; Rust executes
Offline	Limited	Full diagnostics + local narrative
Tool permissions	Admin-defined scripts	Signed repair pack allowlist
Network issues	Separate utility	Fused into agent + dedup

Proactive monitor	Alert only	Handoff to same agent pipeline
Verification	None in chat	Health scan delta in response
Mutating safety	Varies	Chat confirmation + Repair Center gate

8. Request for formal prior-art search

Counsel should deliver:

- 1. Search report with closest references (patents + publications)
- 2. Patentability opinion per application in Document 02
- 3. Claim amendment recommendations
- 4. Freedom-to-operate note for commercial launch

Inventor and Assignment Information Sheet

Complete this form before patent filing. Return to patent counsel with Document 00.

Applicant / assignee

Field	Entry
Legal entity name	
Entity type	☐ Corporation ☐ LLC ☐ Individual ☐ Other
State/country of incorporation	
Mailing address	
Email for USPTO correspondence	

Invention identification

Field	Entry
Invention title	Local-First Autonomous IT Incident Orchestration with Execute-Then-Narrate AI Technician (Jonathan)

Product name	Thorpe Desktop
First conceived (approximate date)	
First reduced to practice (working code)	
First public disclosure (if any)	☐ GitHub ☐ Release ☐ Demo ☐ None — date:
Related applications	☐ None ☐ Provisional #

Inventors

List all individuals who contributed to the **conception** of claimed subject matter. Attorney will finalize inventorship.

Inventor 1

Field	Entry
Full legal name	
Citizenship	
Residence (city, state, country)	
Mailing address	
Email	
Phone	
Contribution summary	e.g., conceived execute-then-narrate architecture, orchestrate_incident design

Inventor 2

Field	Entry
Full legal name	
Citizenship	
Residence	
Contribution summary	

Add additional inventors as needed.

Assignment

Question	Answer
Will inventors assign to applicant entity?	☐ Yes ☐ No
Employment / work-for-hire agreements in place?	☐ Yes ☐ No ☐ Unknown
Contractor contributions?	☐ Yes ☐ No — agreements attached?
Joint development with third party?	☐ Yes ☐ No

Funding and obligations

Source	Obligations?
Self-funded	
Investor funded	
Government grant	☐ Bayh-Dole may apply
University affiliation	

Trademarks (separate from patents)

Mark	Intended use	Filed?
Thorpe	Software / IT support	☐
Jonathan	AI technician persona	☐

Documents enclosed with this package

- [x] 00-PATENT-PACKAGE-INDEX.md
- [x] 01-INVENTION-DISCLOSURE-JONATHAN.md
- [x] 02-PATENT-CANDIDATE-INVENTORY.md
- [x] 03-SYSTEM-ARCHITECTURE-AND-FLOWS.md
- [x] 04-CLAIM-SEEDS-FOR-ATTORNEY.md
- [x] 05-PRIOR-ART-DIFFERENTIATION.md
- [x] 06-INVENTOR-AND-ASSIGNMENT-SHEET.md (this document)
- [] Source code appendix / CD (optional — attorney request)

- [] Screenshots of Jonathan UI (optional)
- [] Signed assignment agreements

Authorized signature

Name	
Title	
Date	
Signature	

Attorney use only

Item	Notes
Docket number	
Provisional filing date	
Non-provisional due date	
Priority claims	
IDS references filed	

Patent Figure and Screenshot Guide

Purpose: Map UI screenshots and diagrams to patent specification figures.

Generated screenshots (screenshots/)

File	Route	Patent figure use	Caption
01-dashboard.png	/	Optional UI context	Thorpe Desktop dashboard with system health summary

02-jonathan-chat-wifi.png	/jonathan	FIG. 6 (UI)	Jonathan assistant after network issue — diagnostics, approval UI
03-jonathan-pending-approval.png	/jonathan	FIG. 7 (UI)	Pending mutating repair approval card with Approve button
04-settings-watchdog.png	/settings	FIG. 8 (UI)	Proactive Watchdog configuration and recent metric alerts
05-repair-center.png	/repairs	Optional	Repair Center with confirmation-gated actions
06-connectivity-report.png	/jonathan	FIG. 9 (UI)	Inline offline connectivity report in chat response

Screenshots are captured from the web preview (mock backend) at 1280×800 using headless Chrome. Attorney may substitute production Tauri captures for filing.

Mermaid diagrams (convert to line drawings)

From 03-SYSTEM-ARCHITECTURE-AND-FLOWS.md:

Figure #	Diagram	Description
FIG. 1	System architecture block	Modules: UI, API, orchestration, AI, DB, OS
FIG. 2	Chat turn sequence	Execute-then-narrate ordering
FIG. 3	Orchestration flowchart	Security, connectivity, plan, filter, verify
FIG. 4	Watchdog handoff sequence	Monitor → notification → Jonathan
FIG. 5	Repair pack gating	Signed manifest → allowlist → execution

Conversion: Export Mermaid to SVG (mermaid.live or npx @mermaid-js/mermaid-cli), then simplify to USPTO black-and-white line art per counsel guidance.

Suggested figure order in specification

- 1. FIG. 1 — System architecture (block diagram)
- 2. FIG. 2 — Method sequence (chat turn)
- 3. FIG. 3 — Orchestration flowchart
- 4. FIG. 4 — Watchdog handoff sequence
- 5. FIG. 5 — Repair pack allowlist
- 6. FIG. 6 — Jonathan chat UI (screenshot)
- 7. FIG. 7 — Repair approval UI (screenshot)

8. FIG. 8 — Watchdog settings (screenshot)

9. FIG. 9 — Connectivity report in chat (screenshot)

Manual capture (Tauri desktop)

If counsel requires native desktop captures:

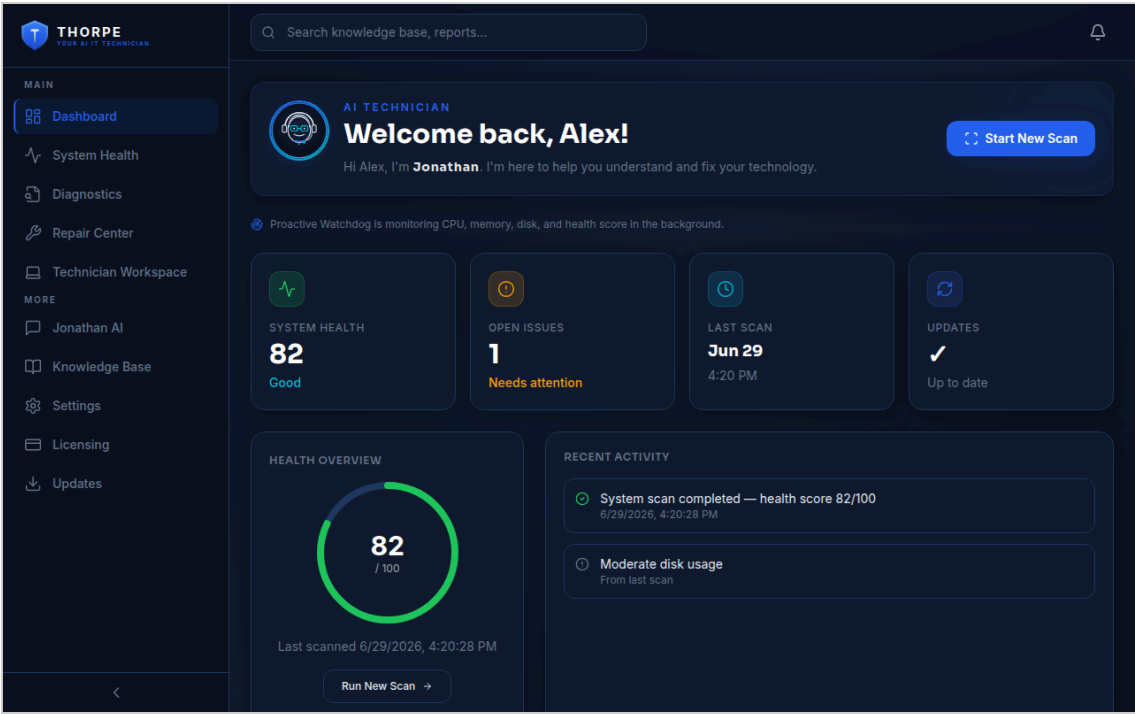
```
bash  
npm run tauri:dev
```

**Navigate to routes above; capture at
1280×800 or 1920×1080**

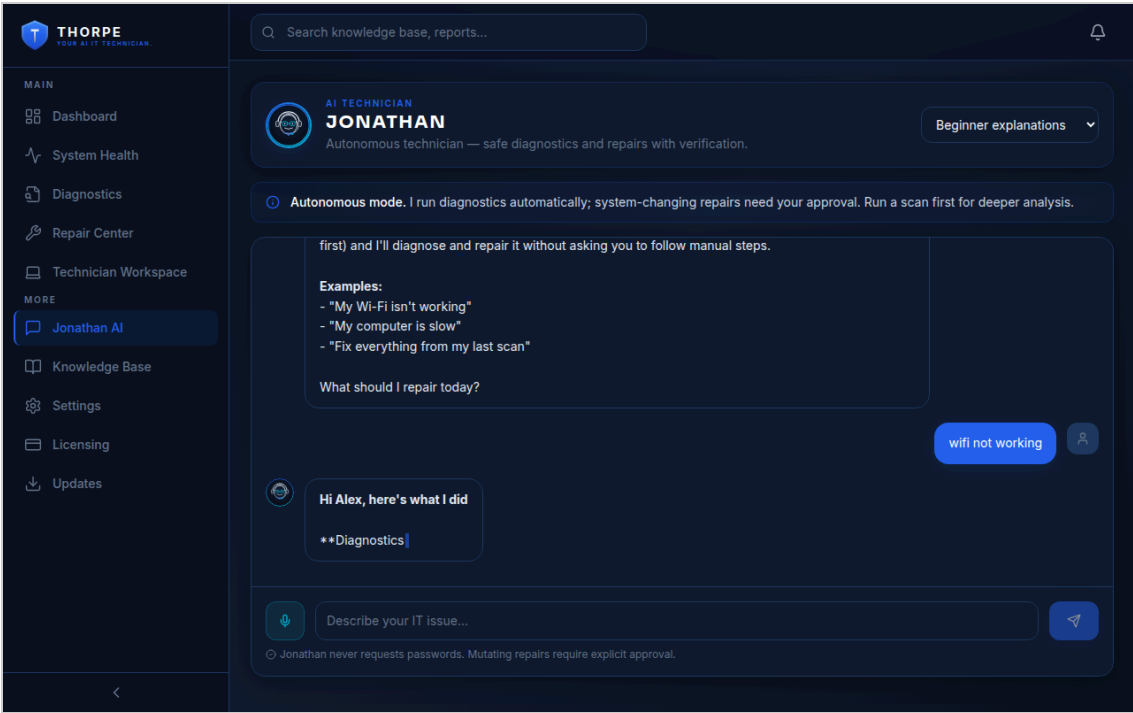
macOS: Cmd+Shift+4 | Windows: Win+Shift+S

Replace files in docs/patents/screenshots/ with same filenames.

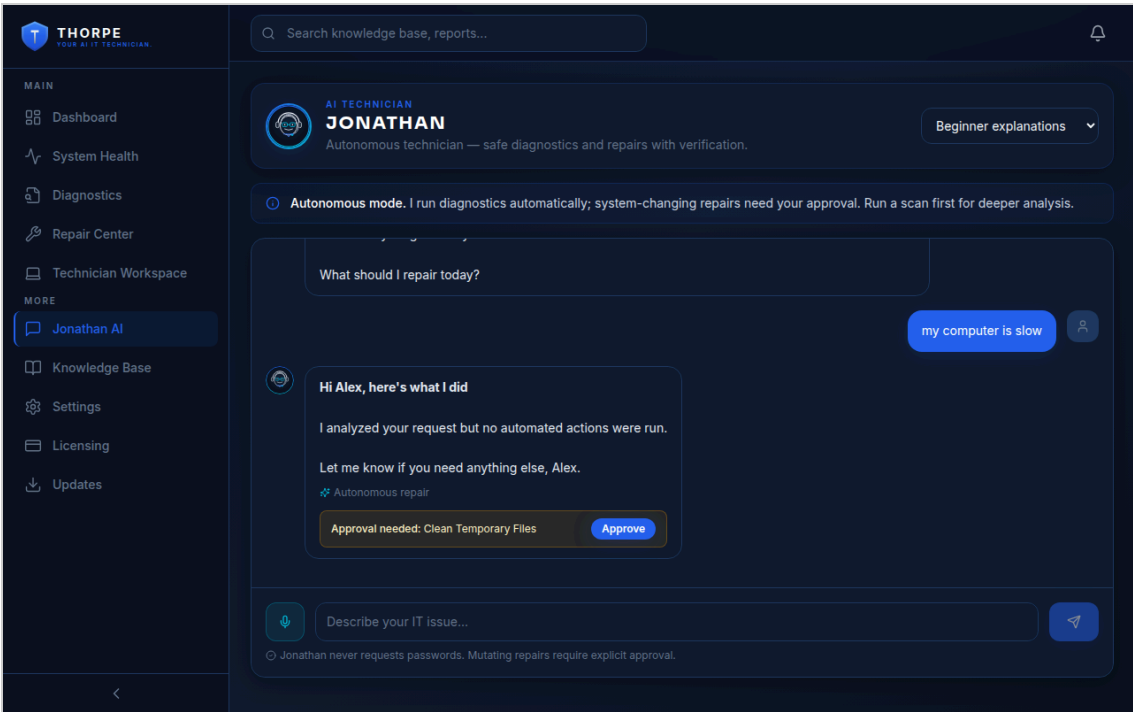
Appendix — UI Screenshots



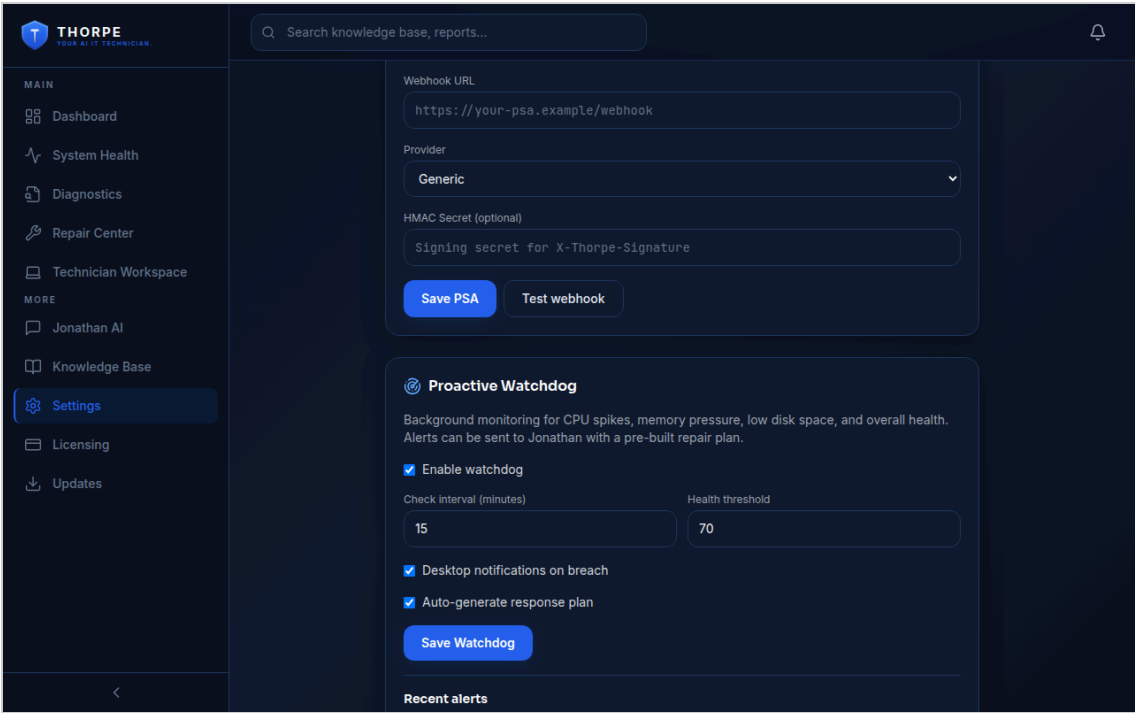
01-dashboard.png



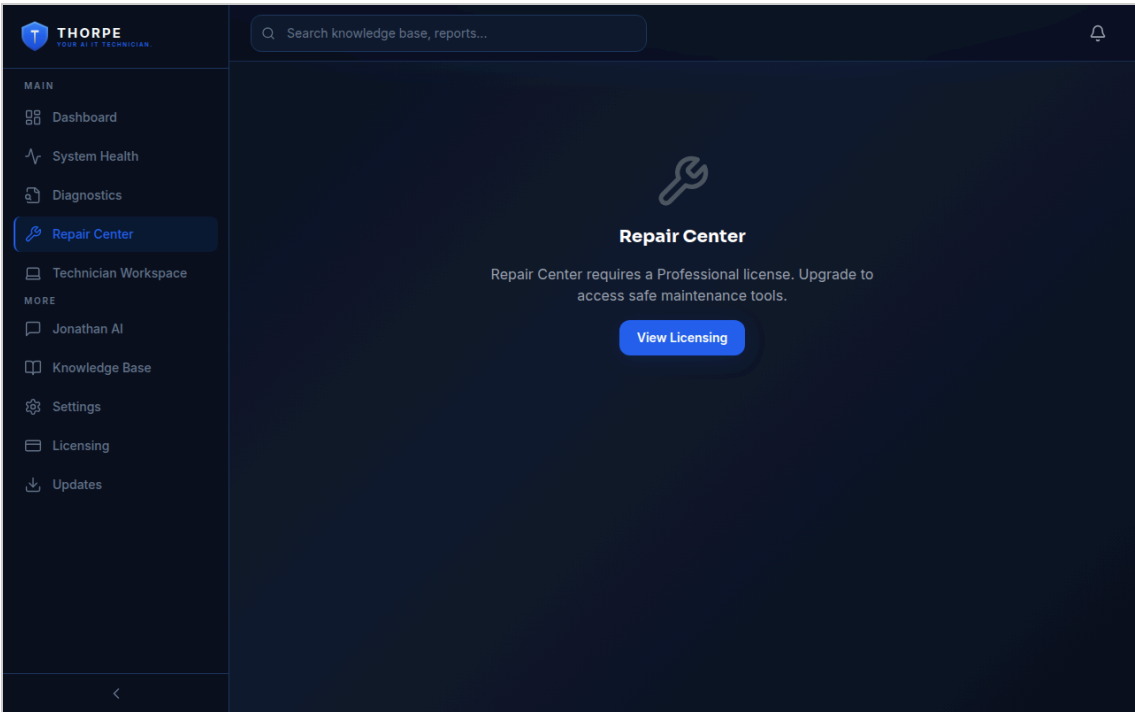
02-jonathan-chat-wifi.png



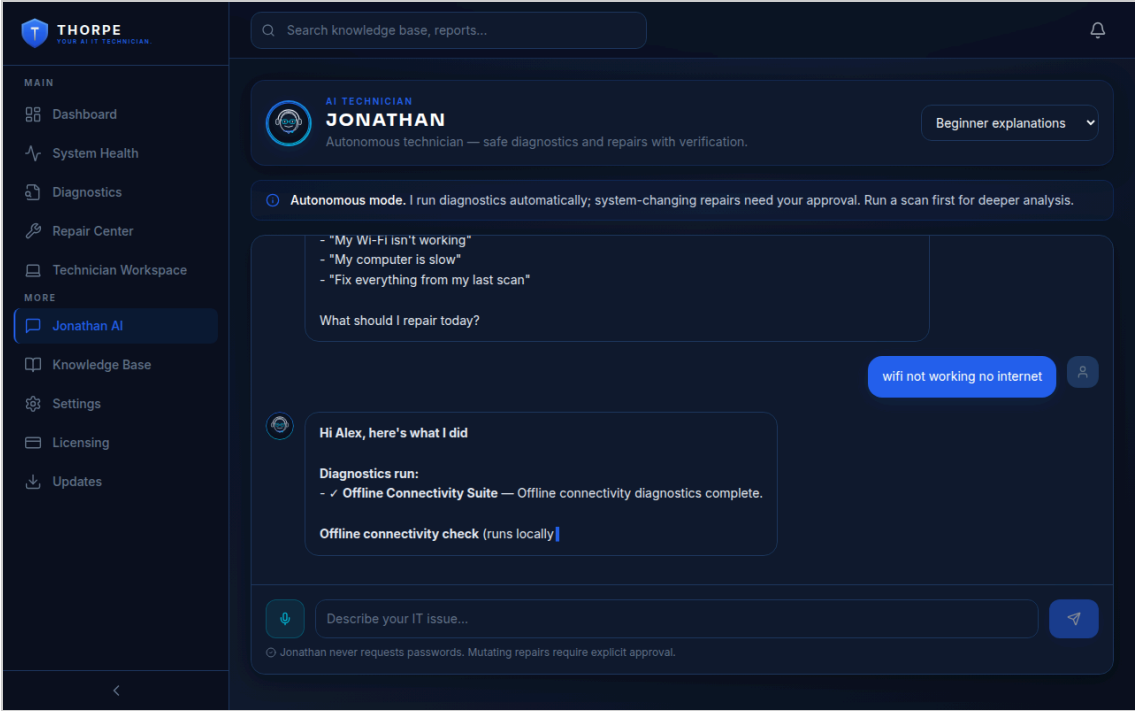
03-jonathan-pending-approval.png



04-settings-watchdog.png



05-repair-center.png



06-connectivity-report.png